On this page

Understand Available Metrics

Access `https://<host:port>/metrics` on your browser to see which metrics are reported. Note that a large number of metrics is reported.

This section explains the different types of metrics being reported and how they are grouped.

Available types of metricsâ

Learn below about the different types of metrics, what kind of information they report, and how they are represented.

Gaugesâ

Gauges are the simplest metric type. They represent a single value.

Example:

```
"audit.writer.activeThreadsCount,metricType=APPLICATION,appType=PULSE_SERVER": {
  "value": 0
}
```

Copy

Countersâ

Counters are similar to gauges, in that they also report a single value. Conceptually, a counter is a simple incrementing and decrementing 64-bit integer.

Example:

```
"pulse.dbPool.pendingThreads,metricType=APPLICATION,appType=PULSE_SERVER,poolName=appsService": {
  "count": 0
}
```

Copy

Metersâ

A meter measures the rate at which a set of events occur. Meters report the following values:

- **count** - The current number of events.
- **m1_rate** - The median rate at which events occur over a 1-minute window.
- **m5_rate** - The median rate at which events occur over a 5-minute window.
- **m15_rate** - The median rate at which events occur over a 15-minute window.
- **mean_rate** - The median rate at which events occur since they have started being recorded.
- **units** - The units of the rate.

Example:

```
"audit.writer.droppedEntries,metricType=APPLICATION,appType=PULSE_SERVER": {
  count: 0,
  m15_rate: 0,
  m1_rate: 0,
  m5_rate: 0,
  mean_rate: 0,
```

```
  units: "events/second"
}
```

Copy

Timersâ□□

A timer is basically a histogram of the duration of a type of event and a meter of the rate of its occurrence.

info

Note that the elapsed times for events are measured internally in nanoseconds, using Javaâ□□s high-precision System.nanoTime() method. Its precision and accuracy vary depending on the operating system and hardware.

Timers report the following values:

- **count** - The current number of events.
- **max** - The maximum duration since this metric has started being reported.
- **mean** - The maximum duration since this metric has started being reported.
- **min** - The minimum duration since this metric has started being reported.
- **stddev** - The standard deviation of the duration since this metric has started being reported.
- **p50** - The duration in the quantile 50.
- **p75** - The duration in the quantile 75.
- **p95** - The duration in the quantile 95.
- **p98** - The duration in the quantile 98.
- **p99** - The duration in the quantile 99.
- **p999** - The duration in the quantile 99.9.
- **m1_rate** - The median rate at which events occur over a 1-minute window.
- **m5_rate** - The median rate at which events occur over a 5-minute window.
- **m15_rate** - The median rate at which events occur over a 15-minute window.
- **mean_rate** - The median rate at which events occur since they have started being recorded.
- **rate_units** - The units of the rate.
- **duration_units** - The units of the duration.

Example:

```
"pulse.dbPool.connectionCreationTimeStats,metricType=APPLICATION,appType=PULSE_SERVER,poolName=a
ppsService": {
  count: 3,
  max: 0,
  mean: 0,
  min: 0,
  p50: 0,
  p75: 0,
  p95: 0,
  p98: 0,
  p99: 0,
  p999: 0,
  stddev: 0,
  m15_rate: 0.05330846541330047,
  m1_rate: 4.86993058876713e-10,
  m5_rate: 0.003787289909588294,
  mean_rate: 0.0024885250809603644,
  duration_units: "seconds",
  rate_units: "calls/second"
}
```

Copy

Tagsâ□□

Before moving on to how to make use of available metrics, itâ□□s also important to understand some additional information that is included in some metrics, which provides information about their context.

Pulse Server and Pulse Applicationsâ□□

A Feedzai Pulse installation is composed of two main services: Pulse Server and Pulse Application. Each service lives in its own JVM process. A Feedzai Pulse installation has one Pulse Server JVM process per node and one additional JVM process for each application, also in the same node.

When you query the `/metrics` endpoint, the retrieved information corresponds to the relevant Pulse Server JVM process and to the JVM processes of each application. Tags are used to distinguish between metrics that correspond to the Pulse Server and to each of the existing Pulse Applications.

Example 1: Pulse Server

```
pulse.dbPool.pendingThreads,metricType=APPLICATION,appType=PULSE_SERVER,poolName=appsService
```

Copy

Notice the **appType** and **poolName** tags. The first one indicates that this metric corresponds to the Pulse Server instance, while the later is used to identify the name of the DB connection pool whose metric is being reported. In this case, the pool name is required as all DB connection pool metrics have the same `pulse.dbPool.pendingThreads` namespace.

Example 2: Pulse Application

```
pulse.dbPool.pendingThreads,appSlug=transactions,metricType=APPLICATION,creator=SimpleRulesTesti
ngReplayModule,appType=PULSE_APPLICATION,poolName=simpleRulesTesting
```

Copy

This metric also corresponds to `pulse.dbPool.pendingThreads` . However, the **appType** tag indicates *PULSE_APPLICATION*, which means that this metric belongs to a Pulse Application. Additionally, the metric contains an *appSlug=transactions* tag. This **appSlug** tag allows you to distinguish between different Pulse Applications running in the same system.

To recap, when monitoring these metrics, consider the following hierarchy:

1. First per **appType**.

   - *PULSE_SERVER* indicates that a metric corresponds to the Pulse Server instance.
   - *PULSE_APPLICATION* indicates that a metric corresponds to a Pulse Application instance.

1. Then per **appSlug**.

   - Where the value is the name of the Pulse Application. This tag is only available when *appType=PULSE_APPLICATION*.

Metric formatâ□□

Metrics are collected using two endpoints. These endpoints collect the same metrics but output different information each:

`/metrics` provides:

- Version
- Metric type
- Metric namespace
- Value

`/metrics/rich` provides:

- Definition
- Description

- Group namespace
- Thread pool name
- Metric name

See below examples of how this information is displayed.

`/metrics` endpoint informationâ□□

Access `https://<host:port>/metrics` on your browser to display the relevant output. To help explain the format of the reported metrics, consider the example below:

```
{
    "version": "3.1.3",
    "gauges":
    {
            "audit.writer.activeThreadsCount,metricType=APPLICATION,appType=PULSE_SERVER":
            {
                    "value": 0
            }
...
```

Copy

What you see above is the first reported metric. Letâ□□s break down its relevant parts:

- *gauges* represents the type of metric being reported. Metrics are grouped by their type. In this case, itâ□□s a gauge and returns a value.
- *audit.writer.activeThreadsCount* is the namespace of the metric. In this case, you are looking at the *activeThreadsCount* of the *audit.writer*.
- *value* is the actual value of the metric. In this case, itâ□□s the number of active threads.

As this example shows, all metrics are represented by their namespaces. However, this information may not be enough to get the actual meaning of each metric. To get more information, you can use the `/metrics/rich` endpoint.

Letâ□□s see how to do this by taking the previous example that used a metric with the *audit.writer.activeThreadsCount* namespace.

`/metrics/rich` endpoint informationâ□□

In the `/metrics/rich` endpoint, while all the same metrics are available, they are grouped differently. Instead of being grouped by type (e.g. the *gauge* above) and represented by their namespace (e.g. *audit.writer.activeThreadsCount* above), they are provided as follows:

```
{
    "definition":
    {
                "description": "The usage ratio of audit writer executor threads and task queue.",
                "evaluators": [ ],
                "name": "audit.writer",
                "tags":
                {
                        "metricType": "APPLICATION",
                        "appType": "PULSE_SERVER"
                },
                "unit": "n/a"
    },
    "metric":
    {
                "metrics":
                {
                        "activeThreadsCount":
                        {
                                    "value": 0
```

```
                    },
...
```

Copy

This is the exact metric as above but represented differently. Letâ□□s break the relevant parts down as before:

- *definition* includes the definition of the group of metrics that monitor a specific module/action.
- *description* explains the group of metrics. In this case, this group reports metrics regarding the thread pool of the audit writer.
- *evaluators* can be ignored as they will not be populated.
- *name* is the groupâ□□s namespace. This is used as the prefix for the namespace of all metrics within the group.
- *tags* allows you to distinguish metrics that have the same namespace from each other. They are used to identify the name of a thread pool.
- *metrics* represents the actual metrics for this group.
- *unit* can be ignored as they will not be populated.
- *activeThreadsCount* is the metric name.

To recap, metrics are represented by their namespace in `/metrics`, and namespaces are composed by `<metric group prefix>.<metric name>`. Letâ□□s apply this logic to our example, where:

- *audit.writer.activeThreadsCount* is the namespace of the metric.
- *audit.writer* is the prefix of the metric group to which the metric belongs.
- *activeThreadsCount* is the name of the metric within the group.

tip

To locate the `audit.writer.activeThreadsCount` metric in `/metrics`, search its group prefix in `/metrics/rich`, and then search its name under that group *(audit.writer)* and *activeThreadsCount*, respectively.